

CURSO DE SQL Y BASES DE DATOS RELACIONALES
Examen Práctico de Consultas SQL

Tienda de Informática ù Empleados ù Ventas

Nombre: _____

Fecha: _____

Grupo: _____

Instrucciones. Cada ejercicio indica su nivel de dificultad: básico ù intermedio ù avanzado. En los ejercicios de opción múltiple solo hay **una** respuesta correcta. En los ejercicios de código, escribe en los espacios marcados con _____. Las soluciones se encuentran al final del documento.

Índice

1. Base de datos: Tienda de Informática

Se trabaja con las tablas *fabricante(id, nombre)* y *producto(id, nombre, precio, id_fabricante)*.

1.1. Consultas sobre una tabla Opción múltiple

Ejercicio 1

¿Cuál de las siguientes consultas lista **únicamente** los nombres de todos los productos de la tabla *producto*?

- a) `SELECT * FROM producto;`
- b) `SELECT nombre FROM producto;`
- c) `SELECT nombre, precio FROM producto;`
- d) `SELECT producto FROM nombre;`

Ejercicio 2

Se quiere mostrar el nombre de los productos con su precio en **céntimos** (multiplicar por 100) usando el alias *centimos*. ¿Qué consulta es correcta?

- a) `SELECT nombre, precio * 100 FROM producto;`
- b) `SELECT nombre, precio / 100 AS centimos FROM producto;`
- c) `SELECT nombre, precio * 100 AS centimos FROM producto;`
- d) `SELECT nombre AS centimos, precio * 100 FROM producto;`

Ejercicio 3

¿Cuál de estas consultas devuelve los fabricantes cuyo nombre **empieza** por la letra S?

- a) `SELECT nombre FROM fabricante WHERE nombre = 'S%';`
- b) `SELECT nombre FROM fabricante WHERE nombre LIKE 'S%';`
- c) `SELECT nombre FROM fabricante WHERE nombre LIKE '%S';`
- d) `SELECT nombre FROM fabricante WHERE nombre IN ('S');`

Ejercicio 4

¿Cuál es el resultado de la siguiente consulta?

```
1 SELECT nombre FROM fabricante ORDER BY nombre DESC LIMIT 2  
   OFFSET 3;
```

- a) Los 2 primeros fabricantes en orden descendente.
- b) Los fabricantes en posiciones 4 y 5 del orden descendente.
- c) Los 3 últimos fabricantes en orden ascendente.
- d) Los fabricantes en posiciones 1 y 2 en orden ascendente.

Ejercicio 5

Se desea listar productos con precio **entre 60 y 200** usando BETWEEN. ¿Qué consulta es equivalente a `WHERE precio >= 60 AND precio <= 200`?

- a) `WHERE precio BETWEEN 60 OR 200`
- b) `WHERE precio BETWEEN 60 AND 200`
- c) `WHERE precio IN (60, 200)`
- d) `WHERE precio > 60 AND precio < 200`

Ejercicio 6

Se necesita listar los productos donde el fabricante sea 1, 3 o 5 usando el operador IN. ¿Cuál es la consulta correcta?

- a) `WHERE id_fabricante = 1 OR 3 OR 5`
- b) `WHERE id_fabricante BETWEEN 1 AND 5`
- c) `WHERE id_fabricante IN (1, 3, 5)`
- d) `WHERE id_fabricante LIKE (1, 3, 5)`

Ejercicio 7

¿Cuál de las siguientes consultas devuelve el nombre y precio del **producto más barato** usando solo ORDER BY y LIMIT?

- a) `SELECT nombre, precio FROM producto ORDER BY precio LIMIT 1;`
- b) `SELECT nombre, MIN(precio)FROM producto;`
- c) `SELECT nombre, precio FROM producto ORDER BY precio DESC LIMIT 1;`
- d) `SELECT nombre, precio FROM producto WHERE precio = MIN(precio);`

1.2. Consultas sobre una tabla Completar código

Ejercicio 8

Completa la consulta para listar los nombres de los fabricantes en orden **descendente**:

```
1 SELECT nombre
2 FROM fabricante
3 ORDER BY ----- ;
```

Ejercicio 9

Completa para mostrar nombre y precio de los productos cuyo precio sea **mayor o igual a 400**:

```
1 SELECT nombre, precio
2 FROM producto
3 WHERE ----- 400;
```

Ejercicio 10

Completa para obtener los fabricantes cuyo nombre tiene **exactamente 4 caracteres**:

```
1 SELECT nombre
2 FROM fabricante
3 WHERE nombre LIKE '-----';
```

Ejercicio 11

Completa la consulta para mostrar las **5 primeras filas** de la tabla fabricante:

```
1 SELECT *
2 FROM fabricante
3 ----- ;
```

Ejercicio 12

Completa la consulta para listar los nombres de los productos convirtiendo el nombre a **mayúsculas**:

```
1 SELECT -----(nombre), precio
2 FROM producto;
```

Ejercicio 13

Completa para obtener 2 filas a partir de la **cuarta fila** de fabricante (la cuarta fila incluida):

```
-----
```

```
1 SELECT *
2 FROM fabricante
3 LIMIT _____ OFFSET _____;
```

1.3. Consultas multitable Opción múltiple

Ejercicio 14

¿Cuál de las siguientes consultas devuelve el nombre del producto, su precio y el nombre del fabricante para **todos los productos**?

a)

```
1 SELECT p.nombre, p.precio, f.nombre
2 FROM producto p, fabricante f;
```

```
1 SELECT p.nombre, p.precio, f.nombre
2 FROM producto p
3 INNER JOIN fabricante f ON p.id_fabricante = f.id;
```

b)

```
1 SELECT p.nombre, p.precio, f.nombre
2 FROM producto p
3 LEFT JOIN fabricante f ON p.id = f.id;
```

d) Las opciones a) y b) son equivalentes y correctas.

Ejercicio 15

Para obtener todos los fabricantes aunque **no tengan productos** asociados, ¿qué tipo de JOIN se debe usar?

- a) INNER JOIN con la tabla fabricante a la izquierda.
- b) LEFT JOIN con fabricante como tabla izquierda.
- c) RIGHT JOIN con producto como tabla izquierda.
- d) CROSS JOIN entre ambas tablas.

Ejercicio 16

¿Cuál de estas consultas devuelve **solamente los fabricantes que NO tienen productos** asociados?

a)

```
1 SELECT f.nombre FROM fabricante f
2 INNER JOIN producto p ON f.id = p.id_fabricante
3 WHERE p.id IS NULL;
```

```
1 SELECT f.nombre FROM fabricante f
```

```

2 LEFT JOIN producto p ON f.id = p.id_fabricante
3 WHERE p.id IS NULL;

```

b)

```

1 SELECT f.nombre FROM fabricante f
2 WHERE f.id NOT IN (SELECT id FROM fabricante);

```

```

1 SELECT f.nombre FROM fabricante f
2 LEFT JOIN producto p ON f.id = p.id_fabricante
3 WHERE p.id IS NOT NULL;

```

1.4. Consultas multitabla Completar código

Ejercicio 17

Completa la consulta para obtener el nombre del producto, precio y nombre de fabricante de **todos los productos del fabricante Lenovo** (usando INNER JOIN):

```

1 SELECT p.nombre, p.precio, f.nombre
2 FROM   producto p
3 ----- fabricante f ON p.id_fabricante = f.id
4 WHERE  f.nombre = -----;

```

Ejercicio 18

Completa la consulta para listar los fabricantes con sus productos, incluyendo fabricantes **sin productos**:

```

1 SELECT f.nombre AS fabricante,
2        p.nombre AS producto
3 FROM   fabricante f
4 ----- producto p ON f.id = p.id_fabricante;

```

Ejercicio 19

Completa la consulta para obtener el nombre del producto más caro del fabricante **Lenovo** usando una subconsulta:

```

1 SELECT nombre
2 FROM   producto
3 WHERE  precio = (
4         SELECT -----(precio)
5         FROM   producto
6         WHERE  id_fabricante = (
7                 SELECT ----- FROM fabricante WHERE nombre = '
                        Lenovo'

```

```
8 )  
9 );
```

Piensa qué función de agregación devuelve el valor máximo. La subconsulta interna recupera el id del fabricante por su nombre.

1.5. Consultas resumen Opción múltiple

Ejercicio 20

¿Cuál de las siguientes consultas calcula correctamente el **número de fabricantes** distintos que tienen productos en la tabla `producto`?

- a) `SELECT COUNT(*)FROM producto;`
- b) `SELECT COUNT(id_fabricante)FROM producto;`
- c) `SELECT COUNT(DISTINCT id_fabricante)FROM producto;`
- d) `SELECT COUNT(DISTINCT nombre)FROM producto;`

Ejercicio 21

Se quiere mostrar el nombre de cada fabricante y el número de productos que tiene, **incluyendo fabricantes sin productos** (con valor 0). ¿Cuál es la estructura correcta?

a)

```
1 SELECT f.nombre, COUNT(p.id) AS total  
2 FROM   fabricante f  
3 INNER JOIN producto p ON f.id = p.id_fabricante  
4 GROUP BY f.nombre ORDER BY total DESC;
```

```
1 SELECT f.nombre, COUNT(p.id) AS total  
2 FROM   fabricante f  
3 LEFT JOIN producto p ON f.id = p.id_fabricante  
4 GROUP BY f.nombre ORDER BY total DESC;
```

b)

```
1 SELECT f.nombre, COUNT(*) AS total  
2 FROM   fabricante f, producto p  
3 GROUP BY f.nombre;
```

```
1 SELECT f.nombre, SUM(p.id) AS total  
2 FROM   fabricante f  
3 LEFT JOIN producto p ON f.id = p.id_fabricante  
4 GROUP BY f.nombre;
```

Ejercicio 22

¿Cuál de las consultas muestra el **precio máximo, mínimo y medio** de los fabricantes cuyo precio medio supera los **200**, mostrando también el nombre del fabricante?

a)

```
1 SELECT f.nombre, MAX(p.precio), MIN(p.precio), AVG(p.precio)
2 FROM producto p
3 INNER JOIN fabricante f ON p.id_fabricante = f.id
4 WHERE AVG(p.precio) > 200
5 GROUP BY f.nombre;
```

```
1 SELECT f.nombre, MAX(p.precio), MIN(p.precio), AVG(p.precio)
2 FROM producto p
3 INNER JOIN fabricante f ON p.id_fabricante = f.id
4 GROUP BY f.nombre
5 HAVING AVG(p.precio) > 200;
```

b)

```
1 SELECT f.nombre, MAX(p.precio), MIN(p.precio), AVG(p.precio)
2 FROM producto p
3 INNER JOIN fabricante f ON p.id_fabricante = f.id
4 GROUP BY f.nombre
5 HAVING COUNT(*) > 200;
```

d) Las opciones a) y b) son equivalentes.

1.6. Subconsultas Completar código

Ejercicio 23

Completa para obtener todos los productos del fabricante **Lenovo** *sin usar* INNER JOIN:

```
1 SELECT *
2 FROM producto
3 WHERE id_fabricante = (
4     SELECT -----
5     FROM fabricante
6     WHERE nombre = 'Lenovo'
7 );
```


Ejercicio 24

Completa la subconsulta para obtener el nombre del producto más caro de toda la tabla, **sin usar** MAX, ORDER BY ni LIMIT (usando ALL):

```
1 SELECT nombre, precio
2 FROM producto
3 WHERE precio >= _____ (
4     SELECT precio FROM producto
5 );
```

ALL devuelve verdadero si la condición se cumple para *todos* los valores de la subconsulta. Si el precio es $\geq$ que *todos* los precios, es el máximo.

Ejercicio 25

Completa la consulta para obtener los fabricantes que **tienen productos** usando EXISTS:

```
1 SELECT f.nombre
2 FROM fabricante f
3 WHERE _____ (
4     SELECT *
5     FROM producto p
6     WHERE p.id_fabricante = _____
7 );
```

2. Base de datos: Gestión de Empleados

Tablas: departamento(id, nombre, presupuesto, gastos) y empleado(id, nif, nombre, apellido1, apellido2, id_departamento).

2.1. Consultas sobre una tabla Opción múltiple

Ejercicio 26

¿Cuál de las siguientes consultas muestra el nombre y apellidos de todos los empleados en **una única columna**?

- a) `SELECT nombre, apellido1 FROM empleado;`
- b) `SELECT CONCAT(nombre, ' ', apellido1, ' ', apellido2)FROM empleado;`
- c) `SELECT nombre + ' ' + apellido1 FROM empleado;`
- d) `SELECT UPPER(nombre, apellido1)FROM empleado;`

Ejercicio 27

Se quiere calcular el **presupuesto actual** de cada departamento (presupuesto inicial menos gastos). ¿Cuál es la consulta correcta?

- a) `SELECT nombre, presupuesto + gastos AS presupuesto_actual FROM departamento;`
- b) `SELECT nombre, presupuesto - gastos AS presupuesto_actual FROM departamento;`
- c) `SELECT nombre, gastos - presupuesto AS presupuesto_actual FROM departamento;`
- d) `SELECT nombre, presupuesto / gastos AS presupuesto_actual FROM departamento;`

Ejercicio 28

¿Cuál de las consultas devuelve los empleados cuyo **segundo apellido** es NULL?

- a) `WHERE apellido2 = NULL`
- b) `WHERE apellido2 = ''`
- c) `WHERE apellido2 IS NULL`
- d) `WHERE ISNULL(apellido2)= 0`

Ejercicio 29

Se quiere listar los departamentos con **gastos mayores que el presupuesto**. ¿Cuál es la consulta adecuada?

- a) `WHERE presupuesto > gastos`
- b) `WHERE gastos > presupuesto`
- c) `WHERE presupuesto = gastos`
- d) `WHERE gastos IS NOT NULL`

2.2. Consultas sobre una tabla Completar código

Ejercicio 30

Completa para listar los apellidos de empleados **sin repetidos**:

```
1 SELECT _____ apellido1
2 FROM empleado;
```

Ejercicio 31

Completa para listar los 3 departamentos con **mayor presupuesto**:

```
1 SELECT nombre, presupuesto
2 FROM departamento
3 ORDER BY -----
4 ----- 3;
```

Ejercicio 32

Completa para obtener 5 filas a partir de la **tercera fila** de empleado (incluida):

```
1 SELECT *
2 FROM empleado
3 LIMIT ----- OFFSET -----;
```

2.3. Consultas multitabla Opción múltiple

Ejercicio 33

¿Cuál de las consultas devuelve un listado con **todos los empleados**, incluyendo los que **no tienen departamento** asignado?

a)

```
1 SELECT e.*, d.nombre FROM empleado e
2 INNER JOIN departamento d ON e.id_departamento = d.id;
```

```
1 SELECT e.*, d.nombre FROM empleado e
2 LEFT JOIN departamento d ON e.id_departamento = d.id;
```

b)

```
1 SELECT e.*, d.nombre FROM departamento d
2 LEFT JOIN empleado e ON e.id_departamento = d.id;
```

```
1 SELECT e.*, d.nombre FROM empleado e, departamento d
2 WHERE e.id_departamento = d.id;
```

Ejercicio 34

Se necesita listar los departamentos **sin empleados** y los empleados **sin departamento** en un solo resultado. ¿Cómo se logra esto en SQL?

- a) Con un único LEFT JOIN de empleado hacia departamento.
- b) Con INNER JOIN y filtrando NULLs.
- c) Con dos consultas unidas por UNION: una con LEFT JOIN (empleados sin departamento) y otra con LEFT JOIN al revés (departamentos sin empleados).

d) No es posible en SQL estándar.

UNION combina resultados de dos SELECT distintos. Puedes hacer un LEFT JOIN en cada dirección y unirlos.

2.4. Consultas resumen Completar código

Ejercicio 35

Completa para calcular el número de empleados en cada departamento, mostrando el **nombre** del departamento y el conteo:

```
1 SELECT d.nombre, COUNT(_____) AS num_empleados
2 FROM departamento d
3 _____ empleado e ON e.id_departamento = d.id
4 GROUP BY _____
5 ORDER BY num_empleados DESC;
```

Ejercicio 36

Completa la consulta para mostrar los departamentos con **más de 2 empleados**:

```
1 SELECT d.nombre, COUNT(e.id) AS total
2 FROM departamento d
3 INNER JOIN empleado e ON e.id_departamento = d.id
4 GROUP BY d.nombre
5 _____ (e.id) > 2;
```

2.5. Subconsultas Opción múltiple

Ejercicio 37

¿Cuál de las consultas devuelve el nombre del departamento con **mayor presupuesto** sin usar MAX, ORDER BY ni LIMIT (usando ALL)?

a)

```
1 SELECT nombre FROM departamento
2 WHERE presupuesto > ALL (SELECT presupuesto FROM
   departamento);
```

```
1 SELECT nombre FROM departamento
2 WHERE presupuesto >= ALL (SELECT presupuesto FROM
   departamento);
```

b)

```
1 SELECT nombre FROM departamento
```

```
2 WHERE presupuesto = ANY (SELECT presupuesto FROM
    departamento);

1 SELECT nombre FROM departamento
2 WHERE presupuesto IN (SELECT MAX(presupuesto) FROM
    departamento);
```

3. Base de datos: Gestión de Ventas

Tablas: *cliente*, *comercial* y *pedido(id, total, fecha, id_cliente, id_comercial)*.

3.1. Consultas sobre una tabla Opción múltiple

Ejercicio 38

¿Cuál de las consultas devuelve todos los pedidos realizados durante el **año 2017** con total superior a **500** ?

a)

```
1 SELECT * FROM pedido WHERE YEAR(fecha) = 2017 AND total
    > 500;
```

```
1 SELECT * FROM pedido WHERE fecha = 2017 AND total >
    500;
```

b)

```
1 SELECT * FROM pedido WHERE fecha LIKE '2017%' OR total
    > 500;
```

```
1 SELECT * FROM pedido WHERE YEAR(fecha) > 2017 AND total
    > 500;
```

Ejercicio 39

¿Cuál de las siguientes consultas devuelve los nombres de los comerciales que **no** aparecen en la tabla *pedido* (usando NOT IN)?

a)

```
1 SELECT nombre FROM comercial
2 WHERE id NOT IN (SELECT id FROM pedido);
```

```
1 SELECT nombre FROM comercial
2 WHERE id NOT IN (SELECT id_comercial FROM pedido);
```

b)

```
1 SELECT nombre FROM comercial
2 WHERE id IN (SELECT id_comercial FROM pedido);
```

```
1 SELECT nombre FROM comercial
2 WHERE id IS NULL;
```

3.2. Consultas sobre una tabla Completar código

Ejercicio 40

Completa para devolver todos los pedidos ordenados por fecha (**más recientes primero**):

```
1 SELECT *
2 FROM    pedido
3 ORDER BY ----- ;
```

Ejercicio 41

Completa para listar los clientes que **no empiezan por A**:

```
1 SELECT nombre
2 FROM    cliente
3 WHERE   nombre ----- LIKE 'A%'
4 ORDER BY nombre;
```

3.3. Consultas multitable Completar código

Ejercicio 42

Completa la consulta para listar todos los clientes junto con sus pedidos, incluyendo clientes **sin pedidos**:

```
1 SELECT c.nombre, c.apellido1, p.total, p.fecha
2 FROM    cliente c
3 ----- pedido p ON c.id = p.id_cliente
4 ORDER BY c.apellido1, c.apellido2, c.nombre;
```

Ejercicio 43

Completa la consulta para mostrar todos los pedidos de la cliente **Adela Salas Díaz** sin usar INNER JOIN:

```
1 SELECT *
2 FROM    pedido
```

```
3 WHERE id_cliente = (  
4     SELECT -----  
5     FROM cliente  
6     WHERE nombre      = 'Adela'  
7         AND apellido1 = 'Salas'  
8         AND apellido2 = -----  
9 );
```

3.4. Consultas resumen Opción múltiple

Ejercicio 44

¿Cuál de las consultas devuelve el **número total de pedidos realizados cada año?**

a)

```
1 SELECT fecha, COUNT(*) FROM pedido GROUP BY fecha;
```

```
1 SELECT YEAR(fecha) AS anio, COUNT(*) AS total  
2 FROM pedido GROUP BY YEAR(fecha) ORDER BY anio;
```

b)

```
1 SELECT YEAR(fecha) AS anio, SUM(total)  
2 FROM pedido GROUP BY anio;
```

```
1 SELECT COUNT(*) FROM pedido WHERE YEAR(fecha) > 0;
```

Ejercicio 45

¿Cuál de las consultas devuelve el número total de pedidos de **cada cliente**, incluyendo los que **no han realizado ningún pedido** (con valor 0)?

a)

```
1 SELECT c.id, c.nombre, COUNT(p.id) AS total  
2 FROM cliente c, pedido p  
3 WHERE c.id = p.id_cliente  
4 GROUP BY c.id;
```

```
1 SELECT c.id, c.nombre, COUNT(p.id) AS total  
2 FROM cliente c  
3 LEFT JOIN pedido p ON c.id = p.id_cliente  
4 GROUP BY c.id, c.nombre;
```

b)

```
1 SELECT c.id, c.nombre, COUNT(*) AS total  
2 FROM cliente c  
3 LEFT JOIN pedido p ON c.id = p.id_cliente  
4 GROUP BY c.id;
```

```
1 SELECT c.id, c.nombre, SUM(p.id) AS total
2 FROM cliente c
3 LEFT JOIN pedido p ON c.id = p.id_cliente
4 GROUP BY c.id;
```

Cuando hay LEFT JOIN y se quiere contar solo pedidos reales (no NULL), usa COUNT(p.id) en lugar de COUNT(*).

3.5. Ejercicio integrador de código completo

Ejercicio 46

Analiza la siguiente consulta e identifica **cuatro errores**:

```
1 SELECT c.nombre, COUNT(*) AS num_pedidos,
2       AVG(p.total) as promedio
3 FROM   cliente c,
4       pedido p
5 WHERE  AVG(p.total) > 500
6 GROUP BY c.nombre, c.id
7 ORDER BY COUNT(*) ASC;
```

Error 1: _____

Error 2: _____

Error 3: _____

Error 4: _____

Piensa en: (1) cómo se hace un JOIN entre tablas, (2) dónde se usan las funciones de agregación para filtrar, (3) qué columna debería estar en GROUP BY, (4) ¿el orden debería ser ASC o DESC para ver los clientes con más pedidos primero?

4. Preguntas de Relacionar Columnas

Ejercicio 47

Relaciona cada cláusula SQL con su descripción correcta:

	Cláusula	Descripción
	WHERE	___ A. Filtra grupos después de agrupar
d)	HAVING	___ B. Elimina filas duplicadas del resultado
	GROUP BY	___ C. Filtra filas antes de agrupar
	DISTINCT	___ D. Agrupa filas con valores iguales
	ORDER BY	___ E. Ordena el resultado

Ejercicio 48

Relaciona cada tipo de JOIN con el resultado que produce:

Tipo JOIN	Resultado
INNER JOIN	___ A. Todas las filas de la tabla derecha + coincidencias
LEFT JOIN	___ B. Solo filas con coincidencia en ambas tablas
RIGHT JOIN	___ C. Todas las filas de la tabla izquierda + coincidencias
UNION	___ D. Resultados de dos SELECT combinados sin duplicados
UNION ALL	___ E. Resultados de dos SELECT combinados con duplicados

Ejercicio 49

Relaciona cada función con su comportamiento frente a valores NULL:

Función	Comportamiento
COUNT(*)	___ A. Ignora NULLs; retorna el primer no-NULL de una lista
COUNT(columna)	___ B. Cuenta todas las filas, incluyendo NULLs
AVG(columna)	___ C. Cuenta solo filas donde la columna no es NULL
COALESCE()	___ D. Ignora NULLs en el cálculo del promedio
IFNULL(a,b)	___ E. Devuelve b si a es NULL; equivalente a COALESCE de 2 args

5. Verdadero / Falso

Ejercicio 50

Indica si cada afirmación es **Verdadera (V)** o **Falsa (F)**:

1. _____ WHERE se evalúa después de GROUP BY.
2. _____ NULL = NULL devuelve TRUE en SQL.
3. _____ DISTINCT elimina filas duplicadas del resultado.
4. _____ TRUNCATE admite cláusula WHERE.
5. _____ Un LEFT JOIN incluye siempre todas las filas de la tabla izquierda aunque no haya coincidencia.
6. _____ HAVING puede usarse sin GROUP BY.
7. _____ COUNT(*) ignora las filas con valores NULL.
8. _____ BETWEEN 60 AND 200 incluye los valores 60 y 200.
9. _____ LIMIT 3 OFFSET 3 devuelve filas 4, 5 y 6.
10. _____ AVG(columna) divide entre el total de filas, incluyendo las que tienen NULL en esa columna.

Soluciones

Solución Sección 1 Tienda de Informática

Opción múltiple: Ej. 1 b) Ej. 2 c) Ej. 3 b) Ej. 4 b) Ej. 5 b) Ej. 6 c) Ej. 7 a)

Completar código:

- Ej. 8: `ORDER BY nombre DESC`
- Ej. 9: `WHERE precio >= 400`
- Ej. 10: `LIKE '____'` (cuatro guiones bajos)
- Ej. 11: `LIMIT 5`
- Ej. 12: `UPPER(nombre)`
- Ej. 13: `LIMIT 2 OFFSET 3`

Multitabla opción múltiple: Ej. 14 d) Ej. 15 b) Ej. 16 b)

Multitabla completar:

- Ej. 17: `INNER JOIN ... 'Lenovo'`
- Ej. 18: `LEFT JOIN`
- Ej. 19: `MAX(precio) ... id`

Resumen opción múltiple: Ej. 20 c) Ej. 21 b) Ej. 22 b)

Subconsultas completar:

- Ej. 23: `id`
- Ej. 24: `ALL`
- Ej. 25: `EXISTS ... f.id`

Solución Sección 2 Empleados

Opción múltiple: Ej. 26 b) Ej. 27 b) Ej. 28 c) Ej. 29 b)

Completar código:

- Ej. 30: `DISTINCT`
- Ej. 31: `presupuesto DESC ... LIMIT`
- Ej. 32: `LIMIT 5 OFFSET 2`

Multitabla opción múltiple: Ej. 33 b) Ej. 34 c)

Resumen completar:

- Ej. 35: `COUNT(e.id) ... LEFT JOIN ... d.nombre`

- Ej. 36: **HAVING** **COUNT**(e.id)> 2

Subconsultas opción múltiple: Ej. 37 b)

Solución Sección 3 Ventas

Opción múltiple: Ej. 38 a) Ej. 39 b)

Completar código:

- Ej. 40: **ORDER BY** fecha **DESC**
- Ej. 41: **NOT LIKE** 'A%' (o **WHERE** nombre **NOT LIKE** 'A%')
- Ej. 42: **LEFT JOIN**
- Ej. 43: id ... 'Díaz'

Resumen opción múltiple: Ej. 44 b) Ej. 45 b)

Ejercicio integrador (Ej. 46) 4 errores:

1. Falta la condición de JOIN en **WHERE**: **WHERE** c.id = p.id_cliente.
2. **AVG**(p.total) > 500 en **WHERE** es incorrecto; las funciones de agregación van en **HAVING**.
3. En **GROUP BY** solo debe aparecer c.id (o c.nombre); no ambos si nombre no es clave (o bien agregar c.nombre de forma consistente).
4. **ORDER BY COUNT(*) ASC** muestra los menos primero; para ver los más frecuentes primero se necesita **DESC**.

Solución Sección 4 Relacionar columnas

Ej. 47 Cláusulas: **WHERE** C **HAVING** A **GROUP BY** D **DISTINCT** B **ORDER BY** E

Ej. 48 JOINS: **INNER JOIN** B **LEFT JOIN** C **RIGHT JOIN** A **UNION** D **UNION ALL** E

Ej. 49 NULLs: **COUNT(*)** B **COUNT(col)** C **AVG** D **COALESCE** A **IFNULL** E

Solución Sección 5 Verdadero / Falso (Ej. 50)

1. **F** **WHERE** se evalúa *antes* de **GROUP BY**.
2. **F** **NULL = NULL** devuelve **NULL**, no **TRUE**. Usar **IS NULL**.
3. **V**
4. **F** **TRUNCATE** no admite **WHERE**; elimina todas las filas.
5. **V**
6. **V** aunque es inusual, es válido sintácticamente.

7. **F** COUNT(*) cuenta *todas* las filas, incluyendo NULLs.
8. **V** BETWEEN es un rango *inclusivo*.
9. **V** OFFSET 3 salta 3 filas; devuelve la 4.ª, 5.ª y 6.ª.
10. **F** AVG ignora los NULLs; divide entre el número de valores no nulos.